Chapter Seven.

Code Optimization and Code Generation

Code Optimization

The code optimization in the synthesis phase is a program transformation technique, which tries to improve the intermediate code by making it consume fewer resources (i.e. CPU, Memory) so that faster-running machine code will result. Compiler optimizing process should meet the following objectives :

- The optimization must be correct, it must not, in any way, change the meaning of the program.
- Optimization should increase the speed and performance of the program.
- The compilation time must be kept reasonable.
- The optimization process should not delay the overall compiling process.

When to Optimize?

Code optimization can be done at various levels of compiling the process.

- At the beginning, users can change/rearrange the code or use better algorithms to write the code.
- After generating intermediate code, the compiler can modify the intermediate code by address calculations and improving loops.
- While producing the target machine code, the compiler can make use of memory hierarchy and CPU registers.

Why Optimize?

Optimization is necessary in the code generated by simple code generator due to the following reasons:

- ❖ Code optimization enhances the portability of the compiler to the target processor.
- ❖ Code optimization allows consumption of fewer resources (i.e. CPU, Memory).
- ❖ Optimized code has faster execution speed.
- ❖ Optimized code gives better performance.
- ❖ Optimized code often promotes re-usability

Types of Code Optimization:

The optimization process can be broadly classified into two types :

1. **Machine Independent Optimization:**

This code optimization phase attempts to improve the **intermediate code** to get a better target code as the output. The part of the intermediate code which is transformed here does not involve any CPU registers or absolute memory locations.

For example:

```
2. do
3. {
4.    item = 10;
5.    value = value + item;
6. } while(value<100);
```

This code involves repeated assignment of the identifier item, which if we put this way:

```
1. Item = 10;
2. do
3. {
4.    value = value + item;
5. } while(value<100);
```

2. **Machine Dependent Optimization:**

Machine-dependent optimization is done after the **target code** has been generated and when the code is transformed according to the target machine architecture. It involves CPU registers and may have absolute memory references rather than relative references. Machine-dependent optimizers put efforts to take maximum **advantage** of the memory hierarchy

- ## Peephole optimization

It is a type of code Optimization performed on a small part of the code. It is performed on a very small set of instructions in a segment of code.

The small set of instructions or small part of code on which peephole optimization is performed is known as **peephole** or **window**.

It basically works on the theory of replacement in which a part of code is replaced by shorter and faster code without a change in output. The peephole is machine-dependent optimization.

The objective of peephole optimization is as follows:

1. To improve performance
2. To reduce memory footprint
3. To reduce code size

Peephole Optimization Techniques

- **Constant folding:** The code that can be simplified by the user itself, is simplified.

  Initial code**:** x = 2 * 3;

  Optimized code**:** x = 6;

- **Strength Reduction:** The operators that consume higher execution time are replaced by the operators consuming less execution time.

Initial code: y = x * 2;

Optimized code: y = x + x ;

Though the output of a * a and a^2 is same, a^2 is much more efficient to implement.

- **Redundant instruction elimination**

```
int add_ten(int x)
  {
  int y, z;
  y = 10;
  z = x + y;
  return z;
  }
```

```
int add_ten(int x)
  {
  int y;
  y = 10;
  y = x + y;
  return y;
  }
```

```
int add_ten(int x)
  {
  int y = 10;
  return x + y;
  }
```

```
int add_ten(int x)
  {
  return x + 10;
  }
```

At compilation level, the compiler searches for instructions redundant in nature. Multiple loading and storing of instructions may carry the same meaning even if some of them are removed. For example:

- MOV x, R0
- MOV R0, R1

We can delete the first instruction and re-write the sentence as: MOV x, R1

- **Unreachable code**

Unreachable code is a part of the program code that is never accessed because of programming constructs. Programmers may have accidently written a piece of code that can never be reached.

Example:

```
void add_ten(int x)
{
  return x + 10;
  printf("value of x is %d", x); //unreachable code
}
```

In this code segment, the **printf** statement will never be executed as the program control returns back before it can execute, hence **printf** can be removed.

❖ **Flow of control optimization**

There are instances in a code where the program control jumps back and forth without performing any significant task. These jumps can be removed. Consider the following chunk of code:

```
...
MOV R1, R2
GOTO L1
...
L1 :  GOTO L2
L2 :  INC R1
```

In this code,label L1 can be removed as it passes the control to L2. So instead of jumping to L1 and then to L2, the control can directly reach L2, as shown below:

```
...
MOV R1, R2
GOTO L2
...
L2 :  INC R1
```

❖ **Loop Unrolling:**

- It helps in optimizing the execution time of the program by reducing the iterations.
- It increases the program's speed by eliminating the loop control and test instructions.

Example: // Before Loop Unrolling

```
for(int i=0;i<2;i++) {
  printf("Hello");
}
//After Loop Unrolling
printf("Hello");
printf("Hello");
```

❖ **Algebraic expression simplification**

There are occasions where algebraic expressions can be made simple. For example, the expression **a = a + 0** can be replaced by **a** itself and the expression a = a + 1 can simply be replaced by INC a.

❖ Code Generation

Code generation can be considered as the final phase of compilation. Through post code generation, optimization process can be applied on the code, but that can be seen as a part of code generation phase itself. The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language. We have seen that the source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties:

- It should carry the exact meaning of the source code.
- It should be efficient in terms of CPU usage and memory management.

❖ Code Generator

A code generator is expected to have an understanding of the target machine's runtime environment and its instruction set. The code generator should take the following things into consideration to generate the code:

- **Target language**: The code generator has to be aware of the nature of the target language for which the code is to be transformed. That language may facilitate some machine-specific instructions to help the compiler generate the code in a more convenient way. The target machine can have either CISC or RISC processor architecture.
- **IR Type**: Intermediate representation has various forms. It can be in Postfix Notation, or 3-address code.
- **Selection of instruction**: The code generator takes Intermediate Representation as input and converts (maps) it into target machine's instruction set. One representation can have many ways (instructions) to convert it, so it becomes the responsibility of the code generator to choose the appropriate instructions wisely.
- **Register allocation**: A program has a number of values to be maintained during the execution. The target machine's architecture may not allow all of the values to be kept in the CPU memory or registers. Code generator decides what values to keep in the registers. Also, it decides the registers to be used to keep these values.
- **Ordering of instructions**: At last, the code generator decides the order in which the instruction will be executed. It creates schedules for instructions to execute them.

- **Descriptors**

The code generator has to track both the registers (for availability) and addresses (location of values) while generating the code. For both of them, the following two descriptors are used:

- **Register descriptor**: Register descriptor is used to inform the code generator about the availability of registers. Register descriptor keeps track of values stored in each register. Whenever a new register is required during code generation, this descriptor is consulted for register availability.
- **Address descriptor**: Values of the names (identifiers) used in the program might be stored at different locations while in execution. Address descriptors are used to keep track of memory locations where the values of identifiers are stored. These locations may include CPU registers, heaps, stacks, memory or a combination of the mentioned locations.

Code generator keeps both the descriptor updated in real-time. For a load statement, LD R1, x, the code generator:

- updates the Register Descriptor R1 that has value of x and
- Updates the Address Descriptor (x) to show that one instance of x is in R1.

In code generation, basic blocks comprise of a sequence of three-address instructions. Code generator takes this sequence of instructions as input.

Note: If the value of a name is found at more than one place (register, cache, or memory), the register's value will be preferred over the cache and main memory. Likewise cache's value will be preferred over the main memory. Main memory is barely given any preference.

- **The getReg Function in code Generation**

Code generator uses ***getReg*** function to determine the status of available registers and the location of name values. *getReg* works as follows:

- If variable Y is already in register R, it uses that register.
- Else if some register R is available, it uses that register.
- Else if both the above options are not possible, it chooses a register that requires minimal number of load and store instructions.

❖ Algorithms for getReg function

For an instruction x = y OP z, the code generator may perform the following actions. Let us assume that L is the location (preferably register) where the output of y OP z is to be saved:

1. Call function getReg, to decide the location of L.
2. Determine the present location (register or memory) of **y** by consulting the Address Descriptor of **y**. If **y** is not presently in register **L**, then generate the following instruction to copy the value of **y** to **L**:
    - MOV **y'**, L
    - where **y'** represents the copied value of **y**.
3. Determine the present location of **z** using the same method used in step 2 for **y** and generate the following instruction:
    - OP z', L
    - where **z'** represents the copied value of **z**.
4. Now L contains the value of y OP z, that is intended to be assigned to **x**. So, if L is a register, update its descriptor to indicate that it contains the value of **x**. Update the descriptor of **x** to indicate that it is stored at location **L**.
5. If y and z has no further use, they can be given back to the system.

Other code constructs like loops and conditional statements are transformed into assembly language in general assembly way.